

OPTION B INSTITUTE FAMILY

M06 - GIT + GITHUB + LINUX/WSL ENGINEERING FOUNDATIONS

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

13 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE06-01 - Verify WSL2 and choose a safe Linux workspace	5
DE06-02 - Linux navigation and safe file operations	7
DE06-03 - Pipes, redirection, command history and inspection	9
DE06-04 - Permissions, packages, sudo and command safety	11
DE06-05 - Git state: init, status, add, commit, diff and log	13
DE06-06 - Branches, merges and real conflict resolution	15
DE06-07 - GitHub remotes: push, fetch, pull and tracking	17
DE06-08 - Pull requests and review workflow	19
DE06-09 - Python virtual environments inside a repository	21
DE06-10 - .gitignore, secrets and repository hygiene	23
DE06-11 - restore, revert, reflog and safe history repair	25
DE06-12 - Shell scripts, exit codes and environment-driven commands	27
DE06-13 - Repository handoff and clean-clone reproducibility	29
Module mastery checklist + revision quick reference	31

Prerequisite / tool map

Work safely in Linux/WSL, version projects with Git/GitHub and hand off reproducible repositories.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE06-01 - Verify WSL2 and choose a safe Linux workspace

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

Outcome Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 21:05-29:43 - Windows/WSL setup and terminal orientation Open assigned video Current/authoritative references: Microsoft Learn - Install WSL SOLO TEACHING BOOK WSL gives a Windows machine a real Linux user space.

5. New words before we use them

Outcome Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 21:05-29:43 - Windows/WSL setup and terminal orientation Open assigned video Current/authoritative references: Microsoft Learn - Install WSL SOLO TEACHING BOOK WSL gives a Windows machine a real Linux user space. First inspect what already exists.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Official guide is intentionally used because WSL setup changes faster than generic videos.

[Open source](#)

7. Follow with me once

PowerShell: `wsl --status`; `wsl -l -v Ubuntu: whoami`; `pwd`; `uname -a`; `mkdir -p ~/de-course/m06` Verify your current WSL state, identify the distro and WSL version, enter the distro, then create `~/de-course/m06/lesson_lab`.

8. Expected result / what you should see

You can show the intended output/state for Verify WSL2 and choose a safe Linux workspace, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If `wsl --install` shows help, inspect existing distros first. Use current Microsoft troubleshooting instead of random manual feature changes. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If `wsl --install` shows help, inspect existing distros first. Use current Microsoft troubleshooting instead of random manual feature changes. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Close the video/book. Prove which terminal you are in, show your Linux home path, create `~/de-course/m06/recheck`, and explain why re-running installation is unnecessary when WSL is already healthy. Validate result A usable distro is visible; Linux commands run inside it; the workspace is under Linux home; your note distinguishes PowerShell/WSL-management commands from Linux commands.

13. Validate

You can show the intended output/state for Verify WSL2 and choose a safe Linux workspace, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Verify WSL2 and choose a safe Linux workspace in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-02

DE06-02 - Linux navigation and safe file operations

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files.

3. Prerequisite / starting state

Prerequisite: DE06-01 mastered with saved evidence.

4. Picture it in your head

Outcome Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 39:05-1:27:51 - pwd, ls, cd, mkdir, touch, rm, mv, cp Open assigned video Current/authoritative references: GNU Coreutils manual SOLO TEACHING BOOK Your current directory changes the meaning of every relative path.

5. New words before we use them

Outcome Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 39:05-1:27:51 - pwd, ls, cd, mkdir, touch, rm, mv, cp Open assigned video Current/authoritative references: GNU Coreutils manual SOLO TEACHING BOOK Your current directory changes the meaning of every relative path. Before a destructive file command, establish location and target with pwd and ls.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Official guide is intentionally used because WSL setup changes faster than generic videos.

[Open source](#)

7. Follow with me once

cd ~/de-course/m06/lesson_lab pwd mkdir -p sandbox/input sandbox/output touch sandbox/input/note.txt cp sandbox/input/note.txt sandbox/output/copy.txt mv sandbox/output/copy.txt sandbox/output/moved.txt ls -la sandbox/output rm sandbox/output/moved.txt Build the sandbox tree, inspect source and destination before every cp/mv/rm, and narrate exactly what changes.

8. Expected result / what you should see

You can show the intended output/state for Linux navigation and safe file operations, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If location or target is uncertain, stop and run pwd plus ls -la. Never use rm -rf to resolve uncertainty. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If location or target is uncertain, stop and run `pwd` plus `ls -la`. Never use `rm -rf` to resolve uncertainty. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Build a different nested tree. Move one named text file through two directories; before deleting it, save `pwd` and `ls` evidence for the exact target directory. Validate result Only the intended sandbox object changes and you can explain absolute vs relative paths plus source vs destination.

13. Validate

You can show the intended output/state for Linux navigation and safe file operations, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Linux navigation and safe file operations in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-03

DE06-03 - Pipes, redirection, command history and inspection

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code.

3. Prerequisite / starting state

Prerequisite: DE06-02 mastered with saved evidence.

4. Picture it in your head

Outcome Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 1:31:56-2:44:04 - head/tail, redirection, cat/less, wc, pipes, sort/uniq, find/grep, history Open assigned video Current/authoritative references: GNU Bash - Redirections GNU Coreutils manual SOLO TEACHING BOOK A pipe sends standard output from one command to the next.

5. New words before we use them

Outcome Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 1:31:56-2:44:04 - head/tail, redirection, cat/less, wc, pipes, sort/uniq, find/grep, history Open assigned video Current/authoritative references: GNU Bash - Redirections GNU Coreutils manual SOLO TEACHING BOOK A pipe sends standard output from one command to the next. > replaces a file; >> appends.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Official guide is intentionally used because WSL setup changes faster than generic videos.

[Open source](#)

7. Follow with me once

`head data/raw/orders.csv grep 'Completed' data/raw/orders.csv | wc -l printf 'east\nwest\nneast\n' | sort | uniq -c python3 src/pipeline.py > evidence.txt echo $? echo 'second note' >> evidence.txt history | tail` Count a known status with `grep|wc`, create a frequency count with `sort|uniq -c`, redirect pipeline output, inspect the exit code, then append a second line without erasing the first.

8. Expected result / what you should see

You can show the intended output/state for Pipes, redirection, command history and inspection, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Run each stage separately. Do not add more pipes while you are unsure which stage changed the result. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

Run each stage separately. Do not add more pipes while you are unsure which stage changed the result. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Use a different status/value from the bundled CSV to build one pipeline and save the result to evidence/inspection.txt. Validate result You can predict where output goes, demonstrate `>` versus `>>`, and break a wrong pipeline into stages to find the defect.

13. Validate

You can show the intended output/state for Pipes, redirection, command history and inspection, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Pipes, redirection, history and text inspection in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-04

DE06-04 - Permissions, packages, sudo and command safety

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts.

3. Prerequisite / starting state

Prerequisite: DE06-03 mastered with saved evidence.

4. Picture it in your head

Outcome Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 4:03:47-5:00:17 - su/sudo, ownership, permissions and chmod Open assigned video Current/authoritative references: Microsoft Learn - WSL file permissions Ubuntu Server - package management SOLO TEACHING BOOK sudo raises privilege for one command; it is not a generic make-it-work prefix.

5. New words before we use them

Outcome Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts. VIDEO FIRST freeCodeCamp - 50 Linux & Terminal Commands | 4:03:47-5:00:17 - su/sudo, ownership, permissions and chmod Open assigned video Current/authoritative references: Microsoft Learn - WSL file permissions Ubuntu Server - package management SOLO TEACHING BOOK sudo raises privilege for one command; it is not a generic make-it-work prefix. chmod changes permission bits.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Official guide is intentionally used because WSL setup changes faster than generic videos.

[Open source](#)

7. Follow with me once

ls -l scripts/run_pipeline.sh chmod u+x scripts/run_pipeline.sh ls -l scripts/run_pipeline.sh # system package metadata only when needed: sudo apt update Inspect permissions, add execute permission only to the bundled training script, and explain why sudo python3 src/pipeline.py is suspicious for a normal user-owned project.

8. Expected result / what you should see

You can show the intended output/state for Permissions, packages, sudo and command safety, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Permission denied? Inspect owner, path and permission bits first. Do not jump straight to sudo or chmod 777. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

Permission denied? Inspect owner, path and permission bits first. Do not jump straight to sudo or chmod 777. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Classify the bundled command-safety worksheet as SAFE / NEEDS EXPLANATION / REJECT and justify target, privilege and consequence. Validate result Before executing a privileged/destructive command you can name the target and expected change; no chmod 777, blind recursive deletion or sudo-as-default is used.

13. Validate

You can show the intended output/state for Permissions, packages, sudo and command safety, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Permissions, packages, sudo and command safety in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-05

DE06-05 - Git state: init, status, add, commit, diff and log

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it.

3. Prerequisite / starting state

Prerequisite: DE06-04 mastered with saved evidence.

4. Picture it in your head

Outcome Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 14:41-16:04 init; 20:20-24:30 status/add; 30:22-32:00 commit; 39:50-40:58 log; 54:26-55:50 diff Open assigned video Current/authoritative references: Git reference SOLO TEACHING BOOK Git has distinct states.

5. New words before we use them

Outcome Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 14:41-16:04 init; 20:20-24:30 status/add; 30:22-32:00 commit; 39:50-40:58 log; 54:26-55:50 diff Open assigned video Current/authoritative references: Git reference SOLO TEACHING BOOK Git has distinct states. Editing changes the working tree; git add proposes content for the next snapshot; git commit records the staged snapshot.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:01:04-0:05:41 + 0:14:41-0:20:20

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

git init git status git add README.md git diff --staged git commit -m "docs: establish project purpose" git log --oneline --decorate -5 Initialize the lesson repository, make two focused changes, stage only one, inspect both unstaged and staged diffs, commit, then inspect history.

8. Expected result / what you should see

You can show the intended output/state for Git state: init, status, add, commit, diff and log, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you are unsure what is staged, do not commit. Use git status, git diff and git diff --staged first. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If you are unsure what is staged, do not commit. Use `git status`, `git diff` and `git diff --staged` first. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Create three simultaneous states: one committed file, one staged change and one unstaged change. Explain each line of `git status` without changing state while explaining. Validate result You can point to the working-tree, index and repository states and use `diff/log` evidence to prove what will or did enter a commit.

13. Validate

You can show the intended output/state for Git state: `init`, `status`, `add`, `commit`, `diff` and `log`, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Git state: `init`, `status`, `add`, `commit`, `diff` and `log` in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-06

DE06-06 - Branches, merges and real conflict resolution

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Create focused branches, merge work and resolve a real text conflict while preserving intended content.

3. Prerequisite / starting state

Prerequisite: DE06-05 mastered with saved evidence.

4. Picture it in your head

Outcome Create focused branches, merge work and resolve a real text conflict while preserving intended content. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 40:58-52:13 - branching, checkout, merge and conflict resolution Open assigned video

Current/authoritative references: Git merge documentation Pro Git - Basic Branching and Merging SOLO TEACHING BOOK A branch is a movable name pointing into commit history.

5. New words before we use them

Outcome Create focused branches, merge work and resolve a real text conflict while preserving intended content. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 40:58-52:13 - branching, checkout, merge and conflict resolution Open assigned video

Current/authoritative references: Git merge documentation Pro Git - Basic Branching and Merging SOLO TEACHING BOOK A branch is a movable name pointing into commit history. A merge combines histories.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:40:58-0:52:13

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

git switch -c feature/docs # edit and commit git switch main # conflicting edit and commit git merge feature/docs # resolve file intentionally git add docs/architecture.md git commit git status Use the bundled conflict fixture to create two incompatible edits to the same line, merge them, resolve to the required combined meaning, and record the final graph.

8. Expected result / what you should see

You can show the intended output/state for Branches, merges and real conflict resolution, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Use git status to list unmerged paths and inspect the file. If confused, stop before destructive reset; the lesson review explains safe abort/retry options. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

Use git status to list unmerged paths and inspect the file. If confused, stop before destructive reset; the lesson review explains safe abort/retry options. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Create a fresh conflict in a different file. Resolve it without deleting unrelated content and explain why keeping only one side blindly is unsafe. Validate result No conflict markers remain; intended content from both changes is preserved where required; git status is clean; history shows the merge.

13. Validate

You can show the intended output/state for Branches, merges and real conflict resolution, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Branches, merges and real conflict resolution in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-07

DE06-07 - GitHub remotes: push, fetch, pull and tracking

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits.

3. Prerequisite / starting state

Prerequisite: DE06-06 mastered with saved evidence.

4. Picture it in your head

Outcome Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 16:04-20:20 - remote repo/clone; 55:50-1:00:24 - push, fetch and pull Open assigned video Current/authoritative references: GitHub Docs - remote repositories Git fetch docs SOLO TEACHING BOOK A remote name such as origin is a local configuration pointing to another repository.

5. New words before we use them

Outcome Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 16:04-20:20 - remote repo/clone; 55:50-1:00:24 - push, fetch and pull Open assigned video Current/authoritative references: GitHub Docs - remote repositories Git fetch docs SOLO TEACHING BOOK A remote name such as origin is a local configuration pointing to another repository. push sends commits/refs; fetch downloads remote state without merging it; pull fetches and then integrates according to configuration.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 0:16:04-0:20:20 + 0:55:50-1:00:24

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`git remote -v` `git push -u origin main` `git fetch origin` `git branch -vv` `git pull --ff-only` Use the bundled local bare remote first: push main, clone it into a second directory, make a remote-side commit, fetch from the first clone, inspect the remote-tracking branch, then integrate deliberately.

8. Expected result / what you should see

You can show the intended output/state for GitHub remotes: push, fetch, pull and tracking, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If histories diverge, inspect git status, branch -vv and log --all --graph before choosing merge/rebase/reset. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If histories diverge, inspect git status, branch -vv and log --all --graph before choosing merge/rebase/reset. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Repeat with a changed remote update. Before pull, prove what fetch changed and what it did not change. Validate result You can distinguish local main, origin/main and the remote repository and explain push vs fetch vs pull.

13. Validate

You can show the intended output/state for GitHub remotes: push, fetch, pull and tracking, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain GitHub remotes: clone, push, fetch, pull and tracking in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-08

DE06-08 - Pull requests and review workflow

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use a feature branch and pull request as a review contract, then synchronize local main after merge.

3. Prerequisite / starting state

Prerequisite: DE06-07 mastered with saved evidence.

4. Picture it in your head

Outcome Use a feature branch and pull request as a review contract, then synchronize local main after merge. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] |

1:16:48-1:19:57 - pull requests and collaboration Open assigned video Current/authoritative references: GitHub Docs - pull requests GitHub Docs - creating a pull request SOLO TEACHING BOOK A pull request is a proposed branch-to-branch change with a diff, discussion and review history.

5. New words before we use them

Outcome Use a feature branch and pull request as a review contract, then synchronize local main after merge. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] |

1:16:48-1:19:57 - pull requests and collaboration Open assigned video Current/authoritative references: GitHub Docs - pull requests GitHub Docs - creating a pull request SOLO TEACHING BOOK A pull request is a proposed branch-to-branch change with a diff, discussion and review history. The PR does not replace Git commits.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: 1:16:48-1:19:57

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

git switch -c feature/readme # edit/test/commit/push # On GitHub: open PR -> inspect Files changed -> merge git switch main git pull --ff-only Prepare a small branch, write a PR-style summary from the diff, and use a local review checklist. If a GitHub account is available, perform the real PR; otherwise save the exact handoff evidence for later account smoke.

8. Expected result / what you should see

You can show the intended output/state for Pull requests and review workflow, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the PR contains unrelated files, do not explain them away. Repair the branch/commits so the diff itself becomes reviewable. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If the PR contains unrelated files, do not explain them away. Repair the branch/commits so the diff itself becomes reviewable. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Create a second tiny branch and produce a reviewable PR packet without copying the first summary. Validate result Diff matches the stated scope; validation is recorded; after merge/simulation local main contains the intended commit and the feature branch is no longer the active work location.

13. Validate

You can show the intended output/state for Pull requests and review workflow, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Pull requests and review workflow in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-09

DE06-09 - Python virtual environments inside a repository

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Create a disposable Python virtual environment in the repository workspace while keeping it out of version control.

3. Prerequisite / starting state

Prerequisite: DE06-08 mastered with saved evidence.

4. Picture it in your head

Outcome Create a disposable Python virtual environment in the repository workspace while keeping it out of version control. VIDEO FIRST Corey Schafer - Python VENV (Mac & Linux) | Full focused venv tutorial - create, activate, use and remove/recreate Open assigned video Current/authoritative references: Python 3.14.7 - venv documentation SOLO TEACHING BOOK A virtual environment isolates a project interpreter/package set.

5. New words before we use them

Outcome Create a disposable Python virtual environment in the repository workspace while keeping it out of version control. VIDEO FIRST Corey Schafer - Python VENV (Mac & Linux) | Full focused venv tutorial - create, activate, use and remove/recreate Open assigned video Current/authoritative references: Python 3.14.7 - venv documentation SOLO TEACHING BOOK A virtual environment isolates a project interpreter/package set. It is disposable, not a deliverable.

6. Useful video / source

Source: Luke Barousse — virtual environments / dependencies

Assigned scope: 6:10:49-6:27:15

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`python3 -m venv .venv`
`source .venv/bin/activate`
`python -c "import sys; print(sys.executable)"`
 deactivate
`git status --ignored --short`
 Create .venv, activate it, prove the Python path is inside the environment, deactivate it and verify Git ignores the directory.

8. Expected result / what you should see

You can show the intended output/state for Python virtual environments inside a repository, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If activation/path is confusing, inspect which python/python3 and sys.executable. Never solve it by committing the environment directory. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If activation/path is confusing, inspect which python/python3 and sys.executable. Never solve it by committing the environment directory. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Delete only the disposable .venv, recreate it from saved instructions and rerun the bundled pipeline. Validate result Project behavior survives environment recreation and Git does not track .venv.

13. Validate

You can show the intended output/state for Python virtual environments inside a repository, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Python virtual environments inside a repository in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-10

DE06-10 - .gitignore, secrets and repository hygiene

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration.

3. Prerequisite / starting state

Prerequisite: DE06-09 mastered with saved evidence.

4. Picture it in your head

Outcome Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 35:32-39:50 - rm and rm --cached; pair with book for .gitignore/secret boundary Open assigned video Current/authoritative references: gitignore documentation GitHub Docs - removing sensitive data SOLO TEACHING BOOK Ignore rules prevent untracked matching files from entering normal Git status, but they do not untrack content already committed.

5. New words before we use them

Outcome Keep secrets, virtual environments, caches and generated runtime outputs out of Git while preserving reproducible examples/configuration. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 35:32-39:50 - rm and rm --cached; pair with book for .gitignore/secret boundary Open assigned video Current/authoritative references: gitignore documentation GitHub Docs - removing sensitive data SOLO TEACHING BOOK Ignore rules prevent untracked matching files from entering normal Git status, but they do not untrack content already committed. A real leaked credential must be revoked/rotated; deleting its current line is not enough.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use repo-hygiene / .gitignore portion; Teaching Book adds secret safety

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
printf '.venv\n.env\n__pycache__\n\nevidence/runtime\n' > .gitignore printf
'API_URL=http://localhost:9999\nTOKEN=replace_me\n' > .env.example git check-ignore -v .env .venv/
|| true git status --ignored --short Create the ignore file and example config, generate fake runtime
files, and prove they are ignored while .env.example remains trackable.
```

8. Expected result / what you should see

You can show the intended output/state for .gitignore, secrets and repository hygiene, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a real credential ever reaches Git history, stop normal practice, revoke/rotate it and follow provider/GitHub cleanup guidance. Do not continue as though deletion solved exposure. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If a real credential ever reaches Git history, stop normal practice, revoke/rotate it and follow provider/GitHub cleanup guidance. Do not continue as though deletion solved exposure. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Introduce a fake secret file into a changed sandbox, catch it before commit using status/diff/check-ignore, and repair the ignore boundary. Validate result No real secret is used; runtime junk is untracked; reproducible examples are committed; you can explain why ignore rules do not erase history.

13. Validate

You can show the intended output/state for .gitignore, secrets and repository hygiene, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain .gitignore, secrets and repository hygiene in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-11

DE06-11 - restore, revert, reflog and safe history repair

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history.

3. Prerequisite / starting state

Prerequisite: DE06-10 mastered with saved evidence.

4. Picture it in your head

Outcome Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 1:00:24-1:03:16 restore; 1:09:47-1:12:18 revert; book adds reflog Open assigned video Current/authoritative references: git restore git revert git reflog SOLO TEACHING BOOK restore operates on working-tree/index content; revert creates a new commit that inverses an earlier commit, making it suitable for shared history; reflog records local reference movements and can help recover a commit name you lost.

5. New words before we use them

Outcome Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 1:00:24-1:03:16 restore; 1:09:47-1:12:18 revert; book adds reflog Open assigned video Current/authoritative references: git restore git revert git reflog SOLO TEACHING BOOK restore operates on working-tree/index content; revert creates a new commit that inverses an earlier commit, making it suitable for shared history; reflog records local reference movements and can help recover a commit name you lost. reset can be useful, but destructive forms are not a default recovery strategy in this course.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use history/repair portion; Teaching Book is authoritative

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

git restore path/to/file git revert <bad-commit> git reflog --oneline -10 git show <recovered-commit>
Make and restore an unstaged mistake, revert a committed mistake, then move a branch reference in a disposable lab and locate the prior commit via reflog.

8. Expected result / what you should see

You can show the intended output/state for restore, revert, reflog and safe history repair, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Before any recovery command, classify the mistake: uncommitted content, staged content, committed/shared history, or lost reference. Then choose the smallest safe tool. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

Before any recovery command, classify the mistake: uncommitted content, staged content, committed/shared history, or lost reference. Then choose the smallest safe tool. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Given three changed incidents, select restore/revert/reflog and justify the choice before running anything. Validate result Final content is correct, history is explainable, and no shared-history rewrite is used as a shortcut.

13. Validate

You can show the intended output/state for restore, revert, reflog and safe history repair, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain restore, revert, reflog and safe history repair in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-12

DE06-12 - Shell scripts, exit codes and environment-driven commands

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes.

3. Prerequisite / starting state

Prerequisite: DE06-11 mastered with saved evidence.

4. Picture it in your head

commands Outcome Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes. VIDEO FIRST freeCodeCamp - Bash Scripting Tutorial for Beginners | 06:21-16:23 scripts/variables/arguments; 16:23-23:23 redirection; 23:23-28:37 conditions; 41:31-42:30 exit codes Open assigned video Current/authoritative references: GNU Bash Reference Manual SOLO TEACHING BOOK A shell script turns a command sequence into a repeatable interface.

5. New words before we use them

commands Outcome Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes. VIDEO FIRST freeCodeCamp - Bash Scripting Tutorial for Beginners | 06:21-16:23 scripts/variables/arguments; 16:23-23:23 redirection; 23:23-28:37 conditions; 41:31-42:30 exit codes Open assigned video Current/authoritative references: GNU Bash Reference Manual SOLO TEACHING BOOK A shell script turns a command sequence into a repeatable interface. Quote variable expansions, validate required inputs before use, return exit code 0 only for success and nonzero for expected failure states.

6. Useful video / source

Source: Microsoft Learn — WSL / Linux environment guide

Assigned scope: Follow the exact current guide + course terminal demo

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Official guide is intentionally used because WSL setup changes faster than generic videos.

[Open source](#)

7. Follow with me once

```
#!/usr/bin/env bash set -u INPUT=${INPUT_FILE:-data/raw/orders.csv} [[ -f "$INPUT" ]] || { echo "missing: $INPUT" >&2; exit 2; } python3 src/pipeline.py "$INPUT" rc=$? exit "$rc"
```

Run the bundled script with its default input, then with INPUT_FILE set to a valid alternative and a missing file. Record stdout/stderr and exit codes.

8. Expected result / what you should see

You can show the intended output/state for Shell scripts, exit codes and environment-driven commands, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If behavior differs between interactive command and script, print only non-secret configuration/path state and inspect exit codes step-by-step. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If behavior differs between interactive command and script, print only non-secret configuration/path state and inspect exit codes step-by-step. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Write a changed runner for the fresh-retest workspace with one required environment variable and two distinct nonzero failure codes. Validate result Success returns 0; missing/invalid inputs return documented nonzero codes; paths containing spaces are safely quoted; no credential value is printed.

13. Validate

You can show the intended output/state for Shell scripts, exit codes and environment-driven commands, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Shell scripts, exit codes and environment-driven commands in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE06-13

DE06-13 - Repository handoff and clean-clone reproducibility

1. Why it matters

This matters because later Git + GitHub + Linux/WSL Engineering Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Prove that another machine/user can recreate and run the project from a clean clone without hidden local state.

3. Prerequisite / starting state

Prerequisite: DE06-12 mastered with saved evidence.

4. Picture it in your head

Outcome Prove that another machine/user can recreate and run the project from a clean clone without hidden local state. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 18:05-20:20 clone + 55:50-1:00:24 remote synchronization; book adds clean-clone acceptance test Open assigned video Current/authoritative references: Git clone documentation GitHub Docs - remote repositories Python venv documentation SOLO TEACHING BOOK A project is not reproducible because it works in the author directory.

5. New words before we use them

Outcome Prove that another machine/user can recreate and run the project from a clean clone without hidden local state. VIDEO FIRST freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 18:05-20:20 clone + 55:50-1:00:24 remote synchronization; book adds clean-clone acceptance test Open assigned video Current/authoritative references: Git clone documentation GitHub Docs - remote repositories Python venv documentation SOLO TEACHING BOOK A project is not reproducible because it works in the author directory. A clean-clone test deliberately starts without untracked files, local virtual environments or shell history.

6. Useful video / source

Source: freeCodeCamp — Git & GitHub Crash Course for Beginners [2026]

Assigned scope: Use clone/push/pull portions; then do clean-clone handoff

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
git clone <remote> clean-check cd clean-check python3 -m venv .venv source .venv/bin/activate
./scripts/run_pipeline.sh git status --short Push/prepare the bundled repository, clone it to a new
directory, recreate its environment, run the pipeline and prove the working tree remains clean except
for explicitly ignored outputs.
```

8. Expected result / what you should see

You can show the intended output/state for Repository handoff and clean-clone reproducibility, name its grain or boundary where relevant, and point to one independent check that confirms it.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the clean clone fails, treat that as a project defect. Identify the hidden dependency, add a safe reproducible instruction/configuration, commit it, clone again and retest. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

11. If you get stuck

If the clean clone fails, treat that as a project defect. Identify the hidden dependency, add a safe reproducible instruction/configuration, commit it, clone again and retest. Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

12. Small independent task

Hand the changed-retest repository to a clean directory and reproduce it using only committed instructions. Record every missing assumption you discover and repair the repository rather than the clone by hand. Validate result A clean clone can be set up and run from documented files; no hidden absolute path or secret is required; validation totals match; repository state is explainable.

13. Validate

You can show the intended output/state for Repository handoff and clean-clone reproducibility, name its grain or boundary where relevant, and point to one independent check that confirms it. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain Repository handoff and clean-clone reproducibility in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.